PRODUCT REQUIREMENTS & SYSTEM ARCHITECTURE

ORBIS

On-Chain Intelligence Infrastructure

Observe. Understand. Reason. Verify.

Document	Product Requirements Document (PRD) & High-Level Architecture Specification
Version	0.1 — Discussion Draft
Date	1 November 2026
Project	Whitechain Builders Program — New Build track (proposed)
Primary architect / builder	Gethsun Misesi
CTO	Bratipah
Status	For review, deliberation and scope approval; not yet an implementation contract

CONFIDENTIAL WORKING DOCUMENT

Prepared for internal project review and prospective Whitechain Builders Program discussions. Network parameters, eligibility, commercial assumptions and funding terms must be validated with Whitechain before external submission.

Document Control

Field	Description
Purpose	Align the project team on ORBIS product scope, architecture, MVP boundaries, interfaces, delivery sequence and open decisions.
Audience	Founder / primary architect, advisory collaborator, prospective technical contributors, Whitechain DevRel and program reviewers.
Status	Draft for deliberation. Requirements and architecture are proposed, not approved implementation commitments.
Version policy	Update version and decision log after each review. Changes to MVP scope, trust boundaries or chain support require explicit approval.
Source of truth	This document becomes the baseline only after the project owners approve the M0 review checklist.

Revision history

Version	Date	Change	Status
0.1	1 Nov 2026	Initial PRD and high-level architecture for review	Draft

How to read this document

Sections 1–6 define the product proposition and requirements. Sections 7–12 describe the proposed architecture, data, interfaces and security model. Sections 13–16 cover delivery, metrics, risks and decisions that must be resolved before implementation. Appendix A records Whitechain source notes and configuration caveats.

Decision discipline

A proposal in this document is not a confirmed Whitechain capability, grant commitment, production service-level agreement or legal opinion. Items marked “validate” remain open until supported by official documentation, DevRel confirmation or a tested deployment.

Contents

1. Executive Summary — Product thesis, value and intended outcome
2. Product Vision & Positioning — Problem, solution, differentiators and ecosystem fit
3. Goals, Non-Goals & Success Criteria — What ORBIS will and will not do
4. Users, Personas & Core Use Cases — Who consumes intelligence and why
5. Product Requirements & MVP Scope — Functional requirements and release boundaries
6. User Journeys & Golden Workflow — End-to-end investigation experience
7. System Architecture — Logical components, deployment boundaries and data flow
8. Data & Knowledge Architecture — Evidence model, persistence and provenance
9. Intelligence & Reasoning Model — Deterministic detection, AI and MeTTa boundaries
10. Developer Interfaces — REST API, SDK, MCP and dashboard
11. Whitechain & Smart-Contract Integration — Network adapter, WBT and on-chain commitments
12. Security, Privacy & Reliability — Trust boundaries, controls and operational requirements
13. Delivery Roadmap & Acceptance Gates — M0–M9 milestones and release criteria
14. Metrics & Commercial Model — Adoption, quality, retention and revenue hypotheses
15. Risks, Dependencies & Mitigations — Technical, product and execution risks
16. Open Decisions & Review Checklist — Items to resolve before M1
Appendix A. Whitechain Reference Notes — Current official references and validation caveats
Appendix B. Glossary — Key terms and abbreviations

Recommended review order: Sections 1–6 first (product agreement), then Sections 7–12 (architecture review), followed by Sections 13–16 (delivery and funding readiness).

1. Executive Summary

ORBIS is a proposed Whitechain-first on-chain intelligence infrastructure platform. It converts raw blockchain activity into persistent, explainable and machine-consumable intelligence for Web3 applications, protocol operators, developers and AI agents.

Rather than building only a monitoring dashboard, ORBIS will expose a shared intelligence core through four interfaces: a REST API, a TypeScript SDK, an MCP server and a human-facing dashboard. External applications will be able to integrate intelligence without independently building their own blockchain indexing, behavioural analysis and evidence-management systems.

Product thesis

Build the infrastructure that allows Web3 applications, developers and AI agents to access, integrate and interrogate reliable on-chain intelligence — with Whitechain as the initial deployment ecosystem.

1.1 Product at a glance

Dimension	Proposed definition
Product	ORBIS — On-Chain Intelligence Infrastructure
Initial network	Whitechain; testnet-first validation, then mainnet deployment when production access and status are confirmed
Primary users	Protocol teams, Web3 startups, developers, liquidity providers and AI-agent builders
Core capabilities	Chain observation, normalized event data, deterministic detections, persistent observations, evidence-linked investigations
Interfaces	REST API / OpenAPI, TypeScript SDK, MCP server, dashboard
Business model	Freemium API and monitoring plans; protocol / enterprise agreements; WBT settlement option subject to legal and commercial validation
Founder model	Solo primary architect and builder, with advisory review from Bratipah
Long-term direction	Reusable EVM intelligence infrastructure; additional networks only after Whitechain product validation

1.2 Intended outcome of the MVP

A user or external application submits a Whitechain contract or wallet address. ORBIS retrieves and normalizes supported on-chain evidence, applies deterministic checks, creates a persistent observation, and returns a structured assessment whose claims link back to transaction hashes, block references, contract addresses and detection-rule versions. AI may explain and correlate the evidence, but it must not invent facts or silently replace deterministic results.

The MVP is successful when this workflow operates against real Whitechain data, is repeatable, is covered by automated tests and can be consumed through a documented API.

2. Product Vision & Positioning

2.1 Problem statement

Blockchain explorers make transactions and contract activity visible, but many applications still lack a reusable layer that converts fragmented events into structured, contextual and auditable intelligence. Teams often assemble their own RPC ingestion, indexing, wallet analytics, rule engines, alert pipelines and AI explanations. This increases engineering cost and produces inconsistent results.

2.2 Proposed solution

ORBIS provides a shared intelligence service that observes supported chains, maintains a normalized evidence store, applies deterministic analytical rules and offers evidence-grounded reasoning through standard developer interfaces.

2.3 Positioning

ORBIS is not intended to be a general-purpose block explorer or a promise of autonomous fraud detection. It is an intelligence infrastructure layer: a service that makes blockchain facts easier to query, relate, assess and integrate into other products.

2.4 Differentiation hypotheses

Dimension	Common monitoring pattern	ORBIS target
Data model	Raw events or isolated alerts	Persistent, normalized observations linked to source evidence
Context	Single event or fixed threshold	Historical context and related-entity analysis where data is available
AI use	Free-form summary	Constrained explanation over retrieved, attributable evidence
Interfaces	Dashboard or proprietary API	API-first, typed SDK, MCP tools and dashboard over one core
Auditability	Alert text with limited provenance	Evidence references, rule versions, timestamps and reasoning trace metadata
Portability	Chain-specific implementation	Provider and chain adapters behind stable internal interfaces

2.5 Whitechain ecosystem fit

Whitechain is the initial target because it is an EVM-compatible network with a native WBT gas asset and a Builders Program designed to support new products through milestone-based support, technical onboarding and ecosystem distribution. ORBIS should demonstrate meaningful Whitechain usage: real indexing, monitoring registrations, developer integrations and, later, verifiable service or evidence transactions.

The product must not treat a chain deployment as sufficient ecosystem commitment. Whitechain-specific product usage and measurable customer activity are the intended outcomes.

Positioning statement

ORBIS turns raw on-chain activity into persistent, explainable and machine-consumable intelligence — enabling developers, protocols and AI agents to integrate blockchain intelligence without building the underlying analytics infrastructure themselves.

3. Goals, Non-Goals & Success Criteria

3.1 Product goals

- **G1 — Reliable observation:** ingest and normalize supported Whitechain blocks, transactions, logs and selected token events.
- **G2 — Evidence-first intelligence:** ensure every material assessment can be traced to underlying on-chain evidence and a versioned rule or analysis method.
- **G3 — Persistent context:** retain observations and entity relationships so later investigations can compare current activity with available history.
- **G4 — Developer-first distribution:** expose stable, documented intelligence through REST API and TypeScript SDK; provide MCP as a subsequent adapter.
- **G5 — Human oversight:** make uncertainty, missing data and analytical limitations visible; never present risk indicators as proof of wrongdoing.
- **G6 — Whitechain commitment:** deploy, test and demonstrate meaningful activity on Whitechain before expanding to additional networks.

3.2 Non-goals for the initial MVP

- Autonomous trading, custody, wallet signing or movement of user funds.
- Guaranteed fraud detection, vulnerability discovery or price prediction.
- A general-purpose blockchain explorer, SQL analytics warehouse or replacement for established explorers.
- A proprietary ORBIS token or complex DAO / tokenomics system.
- Full multichain support, cross-chain identity attribution or universal protocol coverage.
- A custom foundation model trained from scratch.
- On-chain storage of raw user data, private investigation reports or complete evidence payloads.

3.3 MVP success criteria

Area	Initial acceptance target
Chain ingestion	Sustained testnet ingestion with persistent cursor, replay support and duplicate-safe processing.
Data correctness	Every stored event includes chain ID, block number/hash, transaction hash, log index where relevant and source provider metadata.
Detection	At least five deterministic detection rules implemented, tested and documented with known limitations.
Investigation	One complete investigation workflow returns a structured result and traceable evidence references.
API	Versioned REST API documented with OpenAPI and authentication / rate limiting appropriate to the pilot.
Reliability	Retries, backoff, provider health tracking, reorg handling and observable worker failures.
Security	Threat model, secret handling, access controls, dependency checks and contract tests completed before production use.
Adoption	At least one external developer integration or documented pilot; target number to be agreed after outreach.

Numeric SLOs, throughput targets, pricing and customer targets should be set after a baseline load test and pilot interviews. Do not claim service levels before measurement.

4. Users, Personas & Core Use Cases

Persona	Need	ORBIS value
Protocol operator	Understand abnormal contract, token or liquidity activity quickly.	Monitoring, evidence-backed alerts and historical investigations.
Web3 startup / product engineer	Add intelligence without building a full indexing and analytics stack.	Stable API, typed SDK, webhooks and integration documentation.
AI-agent developer	Allow an agent to query trustworthy chain context using standard tools.	MCP tools returning structured data and evidence references.
Liquidity provider / analyst	Understand wallet movements, concentration and pool-related events.	Wallet and protocol assessments with clear caveats.
Ecosystem operator	Observe activity across selected projects and understand ecosystem-level patterns.	Aggregated monitoring and protocol intelligence (later release).

4.1 Primary use cases

UC-01: Assess a contract

A developer submits a contract address and chain. ORBIS retrieves available contract metadata and supported events, evaluates configured rules, and returns a structured profile with evidence links and data-coverage notes.

UC-02: Monitor a wallet

A user registers a wallet for monitoring. ORBIS ingests supported transfer activity, applies configured thresholds and behaviour checks, persists observations and emits alerts when rule conditions are met.

UC-03: Investigate an anomaly

An operator opens an observation. ORBIS retrieves the supporting transactions and historical observations, runs deterministic checks and generates an explanation that distinguishes facts from hypotheses.

UC-04: Integrate via API / SDK

An external application authenticates, requests an assessment, consumes typed results and optionally registers a webhook for new observations.

UC-05: Query through an AI agent

An MCP-compatible client invokes read-only tools to retrieve a wallet profile, contract events, observations or an investigation report. The server returns structured results with provenance and clear limitations.

4.2 User and tenant boundaries

The initial service should support logical tenant isolation even if the first pilot has one operator. API keys, monitoring configurations, alert destinations and private notes must be scoped to a tenant. Public chain facts may be shared; user-specific configurations and reports must not leak across tenants.

5. Product Requirements & MVP Scope

5.1 Functional requirements

ID	Requirement	Priority	Acceptance condition
FR-01	Connect to configured Whitechain RPC provider(s).	Must	Health check returns chain ID, head block and provider status.
FR-02	Ingest blocks and supported logs incrementally.	Must	Cursor advances only after durable processing; replay is safe.
FR-03	Normalize transactions, logs, addresses and token events.	Must	Canonical schema preserves source identifiers and raw evidence references.
FR-04	Detect reorgs and reconcile affected records.	Must	Unfinalized data can be rolled back / replayed without duplicate observations.
FR-05	Persist entities, observations and evidence references.	Must	Records are queryable by chain, address, event type and time.
FR-06	Run deterministic detection rules.	Must	Rule version, inputs, outcome and evidence are persisted.
FR-07	Generate evidence-grounded explanations.	Must	AI output references retrieved evidence and labels hypotheses / uncertainty.
FR-08	Expose versioned REST endpoints.	Must	OpenAPI contract and integration tests pass.
FR-09	Provide API authentication, quotas and rate limits.	Must	Unauthenticated / over-quota requests are rejected consistently.
FR-10	Deliver alert notifications.	Should	Configured Telegram or webhook notification includes observation ID and evidence links.
FR-11	Provide a minimal operator dashboard.	Should	User can search, inspect and review monitoring configurations.
FR-12	Publish a TypeScript SDK.	Should	SDK wraps authentication, typed requests, errors and retries.
FR-13	Expose read-only MCP tools.	Later MVP / M7	MCP tools call the same service layer and enforce tenant / API permissions.
FR-14	Register services and anchor evidence hashes on Whitechain.	Later MVP / M8	Contracts deployed and verified; event and commitment can be independently checked.

5.2 Initial detection catalogue

Rule	Signal	MVP interpretation / limitation
D-01 Large transfer	Transfer value exceeds configured absolute or relative threshold.	Alert only; value threshold is not evidence of malicious intent.
D-02 Mint / burn event	Monitored token emits mint or burn-like Transfer event.	Requires token ABI / event interpretation; non-standard tokens may differ.
D-03 Privileged role change	Ownership or known role-admin event changes.	Coverage depends on contract standard and event visibility.
D-04 Pause / upgrade event	Known pause, unpause, proxy upgrade or implementation change.	Must identify supported standards and avoid assuming every proxy is detected.
D-05 Concentration / accumulation	Observed transfers increase concentration among monitored addresses.	Only computed over indexed and attributed data; no universal wallet identity claim.
D-06 Liquidity movement (candidate)	Pool-specific liquidity withdrawal or reserve change.	Requires supported DEX adapter and pool ABI; include only after protocol selection.

The initial five should be selected based on reliable event coverage and test fixtures. D-06 is a candidate rule, not a promise of universal DeFi pool coverage.

5.3 Non-functional requirements

Category	Requirement
Correctness	Idempotent ingestion; explicit chain identity; preserve raw evidence; deterministic rule execution.
Reliability	Retries with exponential backoff and jitter; dead-letter handling; provider failover; worker health metrics.
Performance	Set measurable ingestion lag and API latency objectives after a testnet baseline.
Scalability	Separate ingestion, analysis and API workloads; queue-based processing; indexed relational queries.
Maintainability	Modular services, typed schemas, migrations, versioned API and automated tests.
Observability	Structured logs, metrics, traces / correlation IDs and alerting on ingestion lag or failures.
Security	Least privilege, secret rotation, tenant isolation, input validation and secure deployment.
Portability	Provider and chain adapters; no Whitechain assumptions embedded in shared domain logic.

6. User Journeys & Golden Workflow

6.1 Primary journey: investigate a Whitechain contract

Step	System action	Expected output
1. Submit	User submits chain identifier and contract address.	Validated assessment request and request ID.
2. Resolve	Observer verifies configured chain, address and available metadata.	Entity record and coverage status.
3. Retrieve	Indexer / RPC adapter fetches supported blocks, logs and token events.	Normalized event set with source references.
4. Detect	Rules engine evaluates configured deterministic checks.	Versioned findings with evidence IDs.
5. Contextualize	Knowledge service retrieves related entities and historical observations.	Context bundle with temporal coverage.
6. Explain	Reasoning service generates constrained explanation from retrieved facts.	Facts, hypotheses, uncertainty and suggested checks.
7. Verify	Evidence service checks references and canonical report digest.	Verification status and report digest.
8. Deliver	API returns assessment; dashboard / webhook / MCP can consume it.	Structured report and evidence links.

6.2 Golden workflow acceptance test

The golden workflow is the release gate for the core product. It must execute end-to-end against real Whitechain testnet data and must not depend on fabricated events or synthetic risk findings.

- Register a known test contract or wallet for observation.
- Ingest a real block and associated transaction / log.
- Persist the event with chain, block and transaction provenance.
- Run a deterministic rule against the event.
- Create an observation with evidence references and rule version.
- Retrieve related historical observations, if available.
- Generate an explanation that cites retrieved evidence and separates facts from hypotheses.
- Return the result through the REST API.
- Verify that the result can be reproduced from stored inputs and rule versions.
- For the later on-chain milestone, anchor the canonical report digest and independently verify it on Whitechain.

Release principle

The system must fail visibly when data is incomplete, a provider is unavailable, a chain is unsupported or evidence cannot be verified. It must not silently substitute fabricated or stale data.

7. System Architecture

The proposed architecture is a modular monolith / service-oriented deployment for the first release: clear internal boundaries and independently testable components, but not an unnecessary fleet of microservices. Split components into separate deployable services only when workload, reliability or team size justifies it.

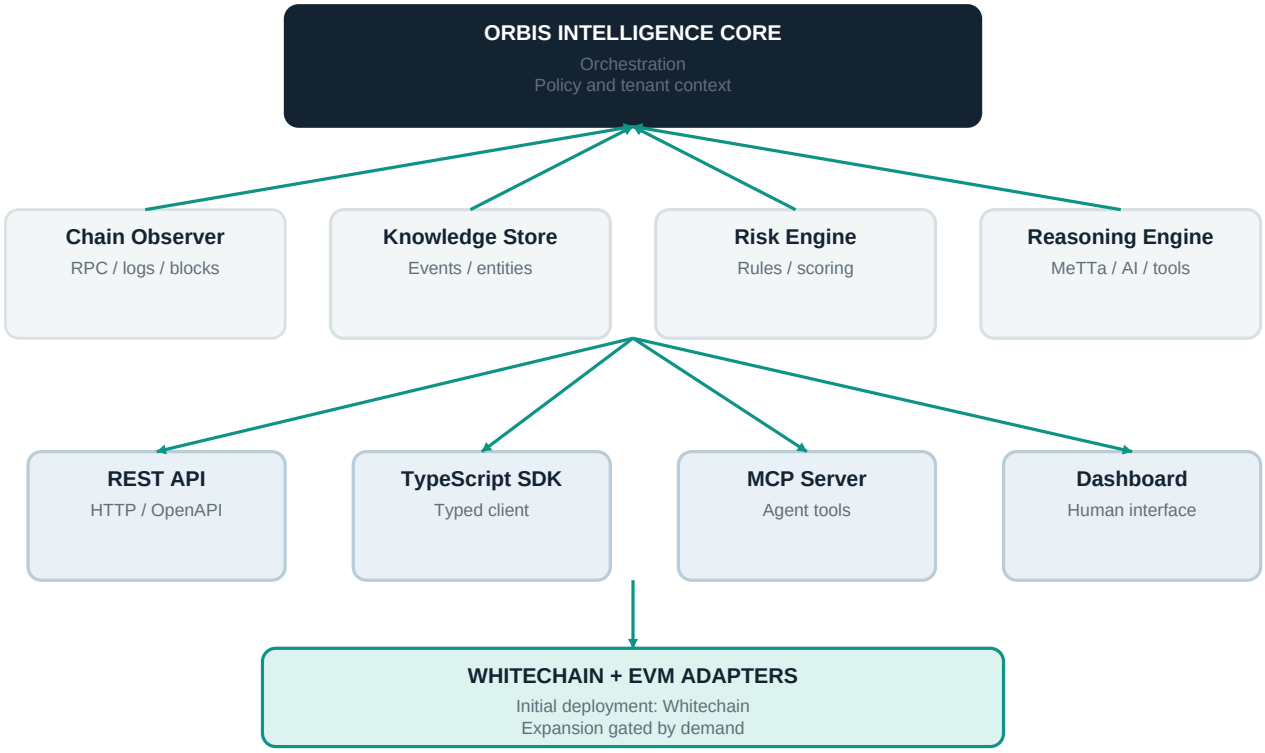


Figure 1. Logical architecture: chain-specific adapters feed a shared intelligence core; developer and user interfaces consume the same service layer.

7.1 Core components

Component	Responsibility	Key boundary
Chain Observer	RPC connectivity, block/log polling, head tracking, reorg detection and provider health.	Does not make risk decisions.
Normalizer	Convert chain-specific payloads into canonical block, transaction, log, token-transfer and entity records.	Preserves raw source identifiers; no lossy inference.
Knowledge Store	Persist chain facts, entities, observations, evidence links and temporal relationships.	PostgreSQL is source of truth for application records.
Risk Engine	Run deterministic, versioned rules and compute explainable indicators.	No LLM-generated facts or hidden score changes.
Reasoning Engine	Retrieve evidence, construct context and produce constrained explanations / investigation plans.	Cannot override deterministic findings or perform privileged writes.
Orchestrator	Coordinate assessment jobs, retries, tool calls and state transitions.	Enforces timeouts, permissions and budgets.
API Service	Authentication, tenant scope, request validation, versioning and response formatting.	Stable public contract; no direct database access from clients.
MCP Server	Expose selected read-only capabilities as MCP tools.	Calls shared service layer; inherits API authorization.
SDK	Typed client wrapper for public API.	No duplicate business logic.
Dashboard / Alerts	Human review, monitoring configuration and notification delivery.	Presentation layer only.

7.2 Proposed technology choices

Layer	Preferred choice	Rationale / caveat
Web application	Next.js + TypeScript	Dashboard and developer portal; keep UI separate from domain logic.
API / workers	Python + FastAPI	Strong fit for data processing, AI orchestration and existing founder experience.
EVM client	web3.py initially; viem for TypeScript tooling where useful	Choose one canonical ingestion client for MVP to reduce duplication.
Contracts	Solidity + Foundry	EVM-compatible toolchain; pin compiler and dependencies to tested versions.
Database	PostgreSQL	Transactional source of truth, JSONB for selected raw payloads, indexed relational data.
Queue / cache	Redis + RQ or Celery	Background jobs and cache; queue state is not the authoritative evidence store.
Reasoning	MeTTa integration behind a reasoning adapter; LLM provider abstraction	Use only where it adds traceable inference; keep deterministic path functional without AI.
MCP	Official MCP SDK / supported transport	Version and security-review the protocol implementation before release.
Deployment	Docker Compose initially; Nginx / TLS; managed or dedicated RPC	Avoid Kubernetes until scale and operational need justify it.
Observability	Structured logs + Prometheus-compatible metrics	Measure ingestion lag, queue depth, API latency and failures.

7.3 Deployment topology

Initial deployment may run as a small set of containers: API, worker / observer, PostgreSQL, Redis, dashboard and reverse proxy. Production readiness requires backups, restore tests, secret management, health checks, resource limits and a documented upgrade / rollback process. The public RPC is suitable for controlled development only unless Whitechain confirms otherwise; production traffic may require a dedicated provider or node.

8. Data & Knowledge Architecture

8.1 Data classes

Class	Examples	Trust treatment
Raw chain evidence	Block headers, tx hashes, logs, receipts, contract addresses.	Source facts; verify against chain and retain identifiers.
Normalized facts	Transfers, role changes, mint/burn events, contract interactions.	Derived representation; retain link to raw evidence and parser version.
Observations	Large transfer, ownership change, unusual event frequency.	Analytical result; include rule ID/version, parameters and evidence.
Hypotheses	Possible coordination, unusual accumulation, potential operational risk.	Unconfirmed; label confidence and supporting / contradicting evidence.
AI explanations	Narrative summary, investigative next steps.	Non-authoritative; cite input observation IDs and model metadata.
User configuration	Watchlists, thresholds, alert endpoints, API keys.	Tenant-private; encrypt sensitive values and apply retention controls.

8.2 Canonical entities

- **Chain:** chain ID, network name, environment, RPC provider configuration and finality policy.
- **Block:** chain ID, block number, block hash, parent hash, timestamp, canonicality / confirmation status.
- **Transaction:** transaction hash, sender, recipient, nonce, value, input selector, status, block reference.
- **Log:** transaction hash, log index, emitting address, topics, data, block reference and decoder version.
- **Entity:** wallet or contract address, chain, observed labels, metadata source and confidence.
- **Observation:** type, severity, status, rule version, time window, evidence references and lifecycle.
- **Assessment:** subject, assessment version, findings, coverage, uncertainty and report digest.
- **Investigation:** question, retrieved context, tool calls, outputs, model metadata and final report.
- **Tenant / API key:** owner, scopes, quotas, creation / expiry / revocation and audit events.

8.3 Illustrative observation object

```
{
"id": "obs_01J...",
"chain_id": 1875,
"subject": {
"type": "contract",
"address": "0x..."
},
"type": "privileged_role_change",
"severity": "high",
"status": "open",
"detector": {
"rule_id": "D-03",
"version": "1.0.0"
},
"observed_at": "2026-11-01T12:00:00Z",
"evidence": [{
"tx_hash": "0x...",
"block_number": 123456,
"block_hash": "0x...",
"log_index": 4
}],
"coverage": {
"from_block": 120000,
"to_block": 123456,
"complete": true
}
}
```

8.4 Evidence provenance

Every normalized record must retain a path back to its chain source. Derived records must identify the parser / decoder version. Rule findings must identify the rule version and parameters. AI-generated explanations must identify the observation IDs and context bundle used. Where reproducibility is required, store a canonical input snapshot or immutable references to all inputs used to generate the result.

8.5 Retention and privacy

Public chain data is inherently public, but watchlists, alert destinations, API credentials, user notes and private reports are not. Store secrets using an appropriate secret-management mechanism; do not log API keys or bot tokens. Establish retention and deletion policies for user-specific data. Do not publish private investigation content on-chain.

9. Intelligence & Reasoning Model

9.1 Three-layer intelligence model

Layer	Function	Authority
Layer 1 — Deterministic analytics	Decode known events, compute metrics, evaluate explicit rules and produce repeatable findings.	Authoritative for the rule result, subject to data completeness and rule correctness.
Layer 2 — Context and knowledge	Retrieve historical observations, connect entities and construct temporal / relational context.	Derived context; must retain provenance and coverage.
Layer 3 — AI-assisted reasoning	Explain findings, compare retrieved patterns, propose follow-up checks and summarize uncertainty.	Advisory only; cannot create chain facts or silently change risk scores.

9.2 Risk scoring principles

Risk scoring should be explainable and decomposable. Each score must expose contributing signals, weights or decision logic, evidence references, rule version and known limitations. Do not present a single number as an objective measure of fraud or safety. During the MVP, categorical severity and rule-level findings may be more defensible than a composite score.

9.3 AI reasoning constraints

- AI receives retrieved evidence and structured observations, not unrestricted authority to infer missing chain facts.
- Responses distinguish observed facts, computed metrics, hypotheses and recommendations.
- Every factual statement in an investigation report should link to an observation or evidence reference where feasible.
- If evidence is insufficient, the model must say so and request additional data or narrow its conclusion.
- Prompt injection contained in contract metadata, token names, logs or external content must be treated as untrusted data.
- AI tool calls are read-only in the MVP; no wallet signing, contract writes or user-fund movement.
- Store model provider, model identifier, prompt / policy version and relevant tool-call metadata for auditability.

9.4 MeTTa / neural-symbolic role

MeTTa may represent entities, observations, predicates and explicit relationships, and support structured inference across persisted knowledge. It should be integrated behind a reasoning adapter so the rest of the platform is not coupled to a single symbolic runtime. The initial release must remain operational if MeTTa or an LLM provider is unavailable: deterministic ingestion, rules and API responses should continue to function.

9.5 Confidence and coverage

Confidence is not the same as probability of maliciousness. It should describe confidence in a particular finding or inference given available evidence and method. Coverage should state what blocks, event types, contracts and time windows were actually inspected. Missing data, unsupported standards and RPC limitations must be visible in the report.

10. Developer Interfaces

10.1 REST API — primary integration contract

The REST API is the canonical public interface. It owns authentication, validation, tenant scope, quotas, versioning and response consistency. All other interfaces should call the same application services.

Proposed endpoint groups

Group	Example routes	Purpose
System	GET /v1/health; GET /v1/chains	Service health and supported network metadata.
Blocks / transactions	GET /v1/blocks/{number}; GET /v1/transactions/{hash}	Read normalized chain facts.
Wallets	GET /v1/wallets/{address}/profile; GET /v1/wallets/{address}/activity	Wallet activity and supported behavioural signals.
Contracts	GET /v1/contracts/{address}/profile; GET /v1/contracts/{address}/events	Contract metadata and observed event history.
Assessments	POST /v1/assessments; GET /v1/assessments/{id}	Create and retrieve evidence-grounded assessments.
Observations	GET /v1/observations; GET /v1/observations/{id}	Search and inspect findings.
Monitoring	POST /v1/monitoring; GET /v1/monitoring; DELETE /v1/monitoring/{id}	Manage watch configurations.
Webhooks	POST /v1/webhooks; DELETE /v1/webhooks/{id}	Notify external systems of events.

Illustrative request

```
POST /v1/assessments
Authorization: Bearer <API_KEY>
Content-Type: application/json

{
"chain": "whitechain",
"subject": {
"type": "contract",
"address": "0x..."
},
"assessment_profile": "contract_baseline",
"time_window": {
"from_block": 100000,
"to_block": "latest"
}
}
```

Illustrative response shape

```
{
"assessment_id": "asmt_01J...",
"status": "completed",
"subject": {"type": "contract", "address": "0x..."},
"coverage": {
"from_block": 100000,
"to_block": 123456,
"complete": true
},
"summary": "Evidence-linked summary...",
"findings": [{
"rule_id": "D-03",
"severity": "high",
"observation_id": "obs_01J...",
```

```
"evidence": [{"tx_hash": "0x...", "block_number": 123456}]
}],
"limitations": [],
"created_at": "2026-11-01T12:00:00Z"
}
```

The endpoint names and schemas are proposals. Final API design must define pagination, idempotency, error codes, async job semantics, rate limits, authentication scopes and backward-compatibility policy.

10.2 TypeScript SDK

Publish a thin, typed client after the REST API stabilizes. The SDK should wrap API authentication, request construction, response types, pagination, retryable errors and timeout configuration. It must not embed risk logic or chain-indexing behaviour.

```
import { OrbisClient } from "@orbis-intelligence/sdk";

const orbis = new OrbisClient({
apiKey: process.env.ORBIS_API_KEY!
});

const assessment = await orbis.assessments.create({
chain: "whitechain",
subject: { type: "contract", address: "0x..." },
assessmentProfile: "contract_baseline"
});
```

10.3 MCP server

MCP is an interoperability adapter for AI-agent clients. Initial tools should be read-only and map directly to existing application services. The MCP server must not bypass API authorization or expose unrestricted RPC, database or shell access.

MCP tool	Purpose	Permission
orbis_get_wallet_profile	Retrieve normalized wallet profile and coverage.	Read
orbis_get_contract_profile	Retrieve contract profile and supported metadata.	Read
orbis_get_transaction	Retrieve transaction facts and receipt evidence.	Read
orbis_search_observations	Search observations for a permitted tenant / subject.	Read
orbis_get_observation	Retrieve finding, evidence and rule metadata.	Read
orbis_assess_subject	Request a bounded assessment through the core service.	Read / compute; quota controlled
orbis_get_investigation	Retrieve completed investigation and provenance.	Read

10.4 Dashboard

The dashboard is an operator and review surface, not the source of truth. MVP screens: overview / system status, monitored entities, entity profile, observation list, observation detail, assessment report, API keys and integration quickstart. Admin-only operational views should be separated from customer-facing views.

11. Whitechain & Smart-Contract Integration

11.1 Network strategy

Whitechain is the first supported network and the reference deployment. Chain-specific configuration must be externalized: chain ID, RPC URLs, explorer base URL, confirmation policy, supported event decoders and provider limits must not be scattered through application code.

Configuration validation required

Official Whitechain sources currently show inconsistent testnet parameters. The wallet configuration page lists Chain ID 2625 and rpc-testnet.whitechain.io, while the Builders page and developer guide identify Whitechain Sepolia as Chain ID 1874 with rpc.testnet.whitechain.io. The project must confirm the canonical current testnet endpoint and chain ID with Whitechain DevRel and by live chain-ID verification before deployment.

11.2 Current reference values — subject to live verification

Environment	Published reference	Status / action
Mainnet	Chain ID 1875; RPC https://rpc.whitechain.io; native asset WBT.	Published by current Whitechain site / docs; verify live RPC and production availability before mainnet deployment.
Testnet / Sepolia	Chain ID 1874 in Builders page and July developer guide; Chain ID 2625 on wallet configuration page.	Conflicting official documentation. Obtain DevRel confirmation and verify eth_chainId before use.
Testnet RPC	https://rpc.testnet.whitechain.io in developer guide; https://rpc-testnet.whitechain.io in wallet configuration page.	Treat both as candidates, not interchangeable confirmed endpoints.

11.3 Indexing strategy

- Use eth_chainId at startup to verify that the endpoint matches configured environment.
- Track block number, block hash and parent hash; persist a durable cursor only after processing is committed.
- Use bounded block-range log queries and adaptive pagination; do not assume unrestricted historical eth_getLogs.
- Implement retry with exponential backoff and jitter; handle HTTP 429 and provider-specific errors.
- Maintain a configurable confirmation depth and reorg rollback / replay process.
- Support multiple providers through an adapter; fail over only when chain identity and data consistency checks pass.
- Record provider and retrieval timestamps for audit and debugging.

11.4 Proposed smart contracts

Contract	MVP role	Design constraints
SentinelRegistry	Register service / monitor metadata or configuration commitments.	Minimal state; events for indexing; avoid storing sensitive customer data.
EvidenceAnchor	Store canonical report digest, schema version and minimal reference metadata.	Hash only; no private reports, raw logs or personal data.
SentinelSubscription	Later: subscription entitlements and WBT settlement.	Defer until pricing, legal review and billing model are validated.

11.5 Evidence commitment

A completed investigation may be serialized using a deterministic canonicalization scheme and hashed. The contract stores the digest, schema version, submitter and timestamp / block context. A verifier compares the digest of the supplied report with the on-chain commitment. A hash proves integrity of the committed bytes; it does not prove the truth or quality of the underlying analysis.

11.6 WBT integration

WBT is the network gas asset. WBT-denominated subscription settlement is a later commercial feature, not a dependency for basic API operation. Keep metering and API authorization off-chain for low latency; use on-chain payments and entitlements as auditable settlement signals. Confirm supported token interface, contract addresses, payment flows and applicable compliance requirements before implementing subscriptions.

12. Security, Privacy & Reliability

12.1 Threat model summary

Threat	Potential impact	Required controls
RPC spoofing / wrong network	Incorrect chain data or cross-environment contamination.	Verify chain ID, expected genesis / checkpoint where available, provider allowlist and consistency checks.
Reorg / non-final data	False or duplicated observations.	Confirmation policy, canonical block tracking, rollback and replay.
Prompt injection in on-chain metadata	Agent manipulation or unsafe tool behaviour.	Treat all chain / external content as untrusted; isolate tool instructions; allowlist tools.
AI hallucination	False or overstated risk conclusions.	Evidence-grounded prompts, structured output validation, fact / hypothesis separation and abstention.
API key theft	Unauthorized access and quota abuse.	Hash / protect keys, scoped permissions, rotation, revocation, rate limits and audit logs.
Tenant data leakage	Exposure of watchlists, private notes or reports.	Tenant scoping at service and database layers; authorization tests.
Webhook abuse / SSRF	Internal network access or data exfiltration.	HTTPS allowlists or destination validation, egress controls, signing and retry limits.
Contract vulnerability	Loss of funds or corrupted registry / commitment state.	Minimal contracts, standard libraries, Foundry tests, static analysis and independent review / audit.
Dependency / supply-chain compromise	Code execution or data compromise.	Pinned dependencies, lockfiles, vulnerability scanning and controlled releases.

12.2 Security requirements

- No private keys or seed phrases are required for read-only monitoring.
- Production secrets must be injected through a secret manager or protected environment; never commit secrets.
- Use least-privilege service accounts and separate development, testnet and production environments.
- Apply request validation, output encoding, authentication, authorization, rate limiting and audit logging.
- MCP tools must be explicitly allowlisted, read-only for MVP and scoped to the caller's authorization.
- AI providers receive only the minimum context necessary; sensitive tenant configuration must not be sent unnecessarily.
- Smart contracts must be tested, fuzzed where appropriate, reviewed and independently audited before handling meaningful value.

12.3 Reliability and data integrity

- Ingestion must be idempotent using stable keys such as chain ID + block hash + transaction hash + log index.
- Queue failures must be retryable and poison messages must be isolated for investigation.
- Database backups and restore procedures must be tested, not merely configured.
- Provider failure must be visible in health endpoints and operational alerts.
- Assessments must record data coverage and freshness; incomplete coverage must not be labelled complete.
- Rule and parser versions must be immutable for historical reproducibility.

12.4 Operational observability

Metric	Why it matters
Indexer head lag	Shows how far ingestion trails the chain head.
Block processing throughput	Measures ingestion capacity and backpressure.
RPC error / 429 rate	Identifies provider constraints and retry pressure.
Queue depth / oldest job age	Shows worker saturation and delayed investigations.
API p50 / p95 latency and error rate	Measures developer experience and reliability.
Assessment duration and AI cost	Controls service economics and timeouts.
Detection volume / false-positive review	Supports rule calibration and user trust.
Webhook delivery success	Measures alert delivery quality.

13. Delivery Roadmap & Acceptance Gates

The schedule below is an indicative solo-founder plan. It is a sequencing proposal, not a commitment to deliver every item within a fixed calendar period. Each milestone ends with a review gate; scope should not advance if the acceptance evidence is missing.

Milestone	Scope	Exit / acceptance evidence
M0 — Product & architecture	Approve PRD, architecture, threat model, API principles, chain configuration and repository strategy.	Signed-off review decisions; unresolved assumptions assigned owners.
M1 — Whitechain observer	RPC adapter, chain verification, block / log ingestion, cursor and provider health.	Real testnet blocks and logs ingested; restart-safe cursor; reorg test.
M2 — Knowledge foundation	Canonical schemas, PostgreSQL persistence, entities, observations and evidence references.	Migrations, indexes, data validation and idempotency tests.
M3 — Deterministic intelligence	Five selected rules, versioning, severity / findings and coverage reporting.	Unit and integration tests with fixtures and known limitations.
M4 — Reasoning & investigations	Context retrieval, constrained AI explanation, MeTTa adapter where justified.	Golden workflow report with evidence references; AI outage does not break core.
M5 — REST API	Auth, endpoints, OpenAPI, pagination, quotas, async assessment jobs.	External developer can complete integration using docs and API key.
M6 — SDK	Typed TypeScript client, errors, retries, pagination and examples.	Published package or release candidate; integration test against API.
M7 — Dashboard & MCP	Operator interface, monitoring configuration and read-only MCP tools.	End-to-end use via dashboard and an MCP-compatible client.
M8 — Whitechain contracts	Registry and evidence anchor; testnet deploy, verification and integration.	Contract tests, verified deployment, event indexing and hash verification.
M9 — Pilot & production readiness	Security review, performance baseline, backups, documentation and pilot onboarding.	Pilot feedback, reliability evidence, production runbook and application demo.

13.1 M0 exit checklist

- Project name and product positioning approved; trademark / domain search assigned.
- Primary users and first supported monitoring subject selected.
- MVP requirements and explicit exclusions approved.
- Whitechain testnet endpoint and chain ID confirmed with DevRel and live RPC check.
- Architecture boundaries and initial technology choices approved.
- Canonical event and observation schema reviewed.
- API authentication, tenancy and read-only policy defined.
- Threat model and evidence provenance requirements accepted.
- M1 tasks broken into small, testable implementation units.
- Repository ownership, licensing and contributor / advisor roles clarified.

14. Metrics & Commercial Model

14.1 North-star metric

Proposed north-star metric

Weekly Active Integrations (WAI): distinct external applications or developer workspaces that successfully consume ORBIS intelligence and return for repeated use within a rolling week.

This metric is more meaningful than raw API calls because retries, bots and one-off testing can inflate request counts. Define an active integration using authenticated successful requests and exclude internal health checks.

14.2 Product and ecosystem metrics

Metric	Definition / measurement
Monitored entities	Distinct wallet / contract subjects with active monitoring configuration.
Indexed chain coverage	Blocks and supported event ranges successfully processed, with gaps reported.
Freshness / lag	Difference between chain head and latest fully processed block.
Assessment completion	Completed assessments divided by valid assessment requests.
Evidence completeness	Share of findings with resolvable source references and coverage metadata.
API active integrations	Distinct external tenants / API keys with meaningful successful use.
SDK adoption	Unique applications using SDK version / package telemetry where opt-in and privacy compliant.
MCP adoption	Distinct authenticated MCP clients / tenants using tools.
Retention	Share of pilot integrations active again after 7 / 30 days.
Commercial conversion	Paying tenants and recurring revenue once billing is launched.
Whitechain activity	Verified contract registrations, evidence commitments and subscription transactions.

14.3 Commercial hypotheses

The proposed model is usage-based API access combined with subscription tiers and negotiated protocol / enterprise plans. Pricing should be validated through customer discovery and cost measurement; no price is approved in this specification.

Offer hypothesis	Potential customer	Value metric
Free developer tier	Independent builders and early startups	Limited monthly requests / monitored entities.
Builder plan	Small Web3 applications	Higher quotas, SDK support and webhooks.
Protocol plan	DeFi / token protocols	Multiple monitored contracts, historical context and team access.
Enterprise / white-label	Established platforms	Dedicated capacity, custom integrations and support.
MCP access	AI agent developers	Metered access to approved intelligence tools.

14.4 WBT and on-chain utility

WBT can be used for network gas and may support subscription settlement after legal, commercial and technical review. The product should not force every API call on-chain. Metering, API keys and request authorization remain off-chain for performance; selected payments, service entitlements and evidence commitments can be represented on-chain.

15. Risks, Dependencies & Mitigations

Risk / dependency	Likelihood / impact	Mitigation / decision
Whitechain RPC limits or endpoint changes	High / High	Provider abstraction, rate-aware pagination, backoff, DevRel confirmation and dedicated RPC option.
Official testnet configuration inconsistency	High / High	Verify chain ID using eth_chainId; confirm canonical endpoint and explorer with DevRel; record date and source.
Insufficient protocol event coverage	Medium / High	Choose a narrow first set of standards / protocols; disclose unsupported patterns.
False positives / weak risk usefulness	High / High	Start with transparent deterministic rules; review pilot findings; tune thresholds with labelled cases.
AI hallucination / prompt injection	Medium / High	Constrained retrieval, untrusted-input handling, schema validation and evidence citations.
Scope expansion by adding SDK, MCP, dashboard and contracts simultaneously	High / High	API-first milestone sequencing; each interface consumes the same core service.
Solo-founder capacity and operational burden	High / High	Modular monolith, automation, clear docs, limited MVP and advisor review cadence.
Grant not awarded / delayed	Medium / High	Build a lean MVP that can continue with staged costs; do not incur grant-dependent obligations prematurely.
Legal / compliance requirements for monitoring and payments	Medium / High	Seek qualified advice; avoid custody, financial advice and unsupported fraud claims.
Brand collision / trademark	Medium / Medium	Search ORBIS and relevant classes / domains before public launch; use ORBIS as working name only.

15.1 Key dependencies

- Whitechain DevRel confirmation of canonical testnet configuration, RPC recommendations and production network readiness.
- Access to real Whitechain testnet activity and a faucet / test assets as required for contract deployment.
- At least one pilot protocol or developer willing to validate the initial intelligence use case.
- Reliable RPC provider or dedicated node path for production-grade indexing.
- Security review and audit plan before contracts hold funds or support significant payment volume.
- Legal / compliance review of service terms, customer data, WBT settlement and any claims made by risk outputs.

Appendix A. Whitechain Reference Notes

The following official sources were consulted while preparing this draft. They should be rechecked immediately before implementation and before any external funding submission because network configuration, program terms and roadmap status can change.

A.1 Builders Program

Whitechain Builders Program — https://whitechain.io/builders

The published page describes a New Build / Builders Program track with milestone-based support and an advertised maximum allocation of up to USD 300,000 per eligible team. It also describes technical support, ecosystem access and a target application decision SLA of two weeks. Funding and launch support are discretionary, subject to eligibility, compliance, review and definitive agreements. Do not represent the maximum allocation or ancillary support as guaranteed.

A.2 Network and developer documentation

Whitechain home / network overview — https://whitechain.io/

The current site identifies Whitechain as an Ethereum L2 / EVM environment and lists mainnet Chain ID 1875 and WBT as the native gas token. Verify the actual production endpoint and current network status before deployment.

Wallet configuration — https://docs.whitechain.io/general/wallet-configuration

The documentation page lists mainnet RPC https://rpc.whitechain.io and Chain ID 1875. Its testnet configuration lists Chain ID 2625 and https://rpc-testnet.whitechain.io.

Start Building on Whitechain — https://whitechain.io/blog/start-building-on-whitechain

The July 30, 2026 developer guide describes Whitechain Sepolia as Chain ID 1874, RPC https://rpc.testnet.whitechain.io and a public testnet RPC limit of 20 requests per second per IP. Treat this as a dated guide; confirm current limits and endpoint before production design.

Smart contract deployment — https://docs.whitechain.io/smart-contracts/smart-contracts-deployment

The deployment guide describes familiar Ethereum tooling including Hardhat, Truffle and Remix, and contract verification through the explorer. Compiler recommendations in the guide should be checked against current supported compiler versions before pinning the project toolchain.

A.3 Required pre-implementation verification

- Ask DevRel to confirm the canonical Whitechain Sepolia chain ID, RPC URL, explorer and faucet.
- Call eth_chainId against the candidate endpoint and compare the returned value to the configured environment.
- Confirm whether mainnet is open for public production deployment and which explorer / RPC are canonical.
- Confirm public RPC rate limits, historical log limits, WebSocket support and production provider recommendations.
- Confirm grant eligibility, milestone evidence, permitted budget categories, audit subsidy terms and required legal documentation.
- Capture date-stamped screenshots or written DevRel confirmation for the application evidence folder.

Appendix B. Glossary

Term	Meaning
Assessment	A bounded evaluation of a wallet, contract or protocol using available data and configured methods.
Canonical block	A block currently considered part of the accepted chain history under the configured finality policy.
Detection rule	A deterministic, versioned procedure that evaluates data and emits a finding.
Evidence reference	A pointer to source data such as transaction hash, block hash, log index or persisted raw payload.
MCP	Model Context Protocol; a standard interface for exposing tools and context to compatible AI applications.
MeTTa	A symbolic / neural-symbolic programming and knowledge representation environment proposed as an optional reasoning component.
Observation	A persisted analytical event or finding generated from chain data and a defined rule.
Reorganisation (reorg)	A change in canonical chain history that may replace previously observed blocks.
SDK	Software Development Kit; a typed client library that simplifies use of the ORBIS API.
Tenant	A customer or workspace boundary used to isolate API keys, monitoring configuration and private data.
WBT	Whitechain's native gas asset, also proposed as an optional settlement asset for future ORBIS subscriptions.

End of document.